

Offline RAG Study Assistant

Implementation Plan & Technical Documentation

Theme: On-Device AI | Team Size: 3 | Format: Hackathon Build Plan

This document is a complete implementation reference for building an **Offline RAG (Retrieval-Augmented Generation) Study Assistant** that runs entirely on-device, in the browser, using WebGPU-accelerated local LLM inference. No document content, embeddings, or generated answers ever leave the user's device. Cloud services are used only for optional non-AI support features (auth, cross-device metadata sync).

Purpose of this document: This file is written to be uploaded as project context to a local LLM (e.g. via Ollama) to assist with code generation, debugging, and architectural decisions during the build. It is structured with explicit component boundaries, file paths, and code stubs so a model can reference it directly.

Compliance: This plan is aligned to the **OSDHack 2026 Resource Guide** (Open Source Developers Community). It follows the event's core rule — main AI functionality runs locally/on-device/in-browser — and its "Models vs. Libraries" distinction: every model used below is a specific, named, downloadable weights artifact (not a bare architecture import). See Section 19 for the filled-in per-model documentation table required by the rulebook.

Contents

- 1. Problem Statement & Goals
- 2. System Architecture
- 3. Tech Stack
- 4. Data Flow
- 5. Project Folder Structure
- 6. Component 1 — Document Ingestion & Chunking
- 7. Component 2 — Embedding & Vector Store
- 8. Component 3 — On-Device LLM (WebLLM)
- 9. Component 4 — RAG Orchestration
- 10. Component 5 — UI / Chat Interface
- 11. Optional Cloud Layer (Supabase)
- 12. Deployment Guide
- 13. Team Role Split (3 People)
- 14. Build Timeline
- 15. Testing Checklist
- 16. Demo Script for Judges
- 17. Known Risks & Mitigations
- 18. Appendix — Prompt Templates & Config
- 19. OSDHack 2026 Compliance & Model Documentation

1. Problem Statement & Goals

Students need to query their own notes/PDFs conversationally, but existing AI study tools require an internet connection and send private study material to a third-party server. This is a problem for students in low-connectivity areas, on flights, or anyone who does not want personal notes uploaded externally.

Goals

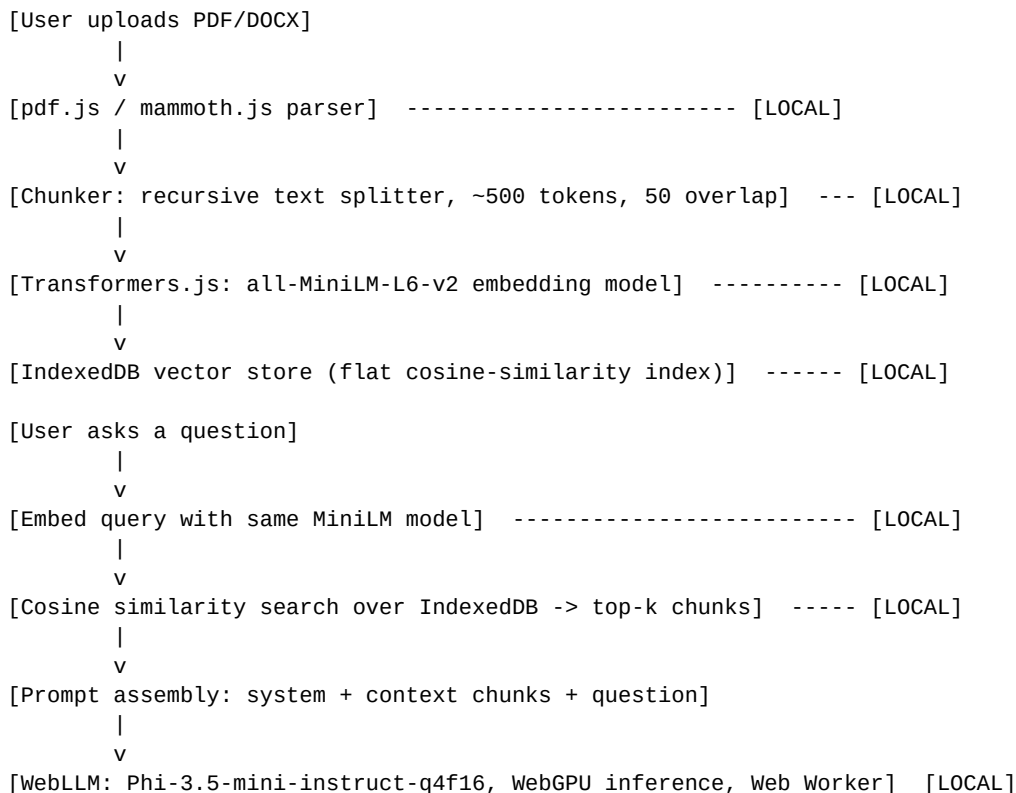
- Upload PDF/DOCX notes and have them chunked and embedded entirely in-browser.
- Ask natural-language questions and get answers generated by a local LLM running via WebGPU, with zero network calls during inference.
- Prove the offline claim live: disconnect the network and continue functioning identically.
- Keep the app usable on a normal laptop (integrated GPU, 8-16GB RAM) without requiring a dedicated GPU.

Non-goals (explicitly out of scope for hackathon build)

- Multi-user collaboration / shared documents.
- Full mobile app (browser-based only for this build).
- Fine-tuning or training any model.

2. System Architecture

High-level flow, all boxes marked [LOCAL] execute entirely in the browser with no network dependency:



|
v
[Streamed answer rendered in chat UI]

OPTIONAL CLOUD LAYER (non-AI, support only):
[Supabase Auth] -> user login
[Supabase Postgres] -> stores document TITLES + timestamps only
 (never document content, never embeddings,
 never chat history)

3. Tech Stack

Layer	Choice	Reasoning
LLM inference	WebLLM (MLC-AI) Phi-3.5-mini-instruct-q4f16	WebGPU-accelerated, runs fully in-browser, good reasoning-per-MB at ~2.2
Fallback LLM	Llama-3.2-1B-Instruct-q4f16	~800MB, guarantees it runs on weak/unknown demo hardware
Embeddings	Transformers.js Xenova/all-MiniLM-L6-v2 (ONNX)	~25MB, fast, standard for RAG, runs on CPU if no WebGPU
Vector store	IndexedDB + flat cosine search (or client-vector-search npm pkg)	No server needed; brute-force search is fine at hackathon scale (few thousa
PDF parsing	pdf.js	Mozilla's battle-tested client-side PDF text extraction
DOCX parsing	mammoth.js	Client-side docx-to-text
Frontend	React + Vite + Tailwind	Fast to build, team already familiar
Worker model	Web Worker for WebLLM	Keeps UI responsive during inference
Optional cloud	Supabase (auth + metadata only)	Free tier, fast to wire up, never touches AI compute
Hosting	Vercel or Netlify	Static hosting, free tier, HTTPS by default (required for WebGPU)

Note on Ollama: Ollama itself is a desktop-only runtime and is NOT used inside the deployed web app (it cannot run inside a browser tab). It is used here only as the local model you are consulting while building this project. If you later want a desktop-app version instead of browser-based, Ollama + a local Electron/Tauri shell would replace WebLLM — see section 17.

4. Data Flow

4.1 Ingestion flow

- File dropped/selected in UI → read as ArrayBuffer.
- pdf.js/mammoth.js extracts raw text.
- Chunker splits text into ~500-token chunks with 50-token overlap, tagging each chunk with source filename + page number.
- Each chunk is embedded via Transformers.js → 384-dim vector.
- Chunk text + vector + metadata stored as a record in IndexedDB.

4.2 Query flow

- User types a question in chat UI.
- Question is embedded with the same MiniLM model.
- Cosine similarity computed against all stored chunk vectors; top-k (k=4-6) retrieved.
- Retrieved chunks + question inserted into a prompt template (see Appendix 18.1).
- Prompt sent to WebLLM running in a Web Worker; tokens streamed back to UI as they generate.
- Answer rendered with a small citation tag showing which chunk(s)/page(s) it came from.

5. Project Folder Structure

```
offline-rag-study-assistant/  
  ■■■ public/  
  ■■■ src/  
    ■ ■■■ components/  
    ■ ■ ■■■ ChatWindow.jsx  
    ■ ■ ■■■ DocUploader.jsx  
    ■ ■ ■■■ MessageBubble.jsx  
    ■ ■ ■■■ OfflineIndicator.jsx  
    ■ ■ ■■■ ModelSelector.jsx  
    ■ ■■■ lib/  
    ■ ■ ■■■ parsing/  
    ■ ■ ■■■ ■■■ pdfParser.js  
    ■ ■ ■■■ ■■■ docxParser.js  
    ■ ■ ■■■ chunking/  
    ■ ■ ■■■ ■■■ chunker.js  
    ■ ■ ■■■ embeddings/  
    ■ ■ ■■■ ■■■ embedder.js (Transformers.js wrapper)  
    ■ ■ ■■■ vectorstore/  
    ■ ■ ■■■ ■■■ indexedDbStore.js  
    ■ ■ ■■■ llm/  
    ■ ■ ■■■ ■■■ webllmWorker.js (runs inside Web Worker)  
    ■ ■ ■■■ ■■■ webllmClient.js (main-thread interface)  
    ■ ■ ■■■ rag/  
    ■ ■ ■■■ ■■■ ragPipeline.js (ties retrieval + generation together)  
    ■ ■■■ App.jsx  
    ■ ■■■ main.jsx
```

```
■■■ package.json
■■■ vite.config.js
■■■ index.html
```

6. Component 1 — Document Ingestion & Chunking

Owner: Person A. Responsible for turning uploaded files into clean, chunked text ready for embedding.

pdfParser.js

```
import * as pdfjsLib from 'pdfjs-dist';

export async function parsePdf(file) {
  const buf = await file.arrayBuffer();
  const pdf = await pdfjsLib.getDocument({ data: buf }).promise;
  const pages = [];
  for (let i = 1; i <= pdf.numPages; i++) {
    const page = await pdf.getPage(i);
    const content = await page.getTextContent();
    const text = content.items.map(it => it.str).join(' ');
    pages.push({ pageNumber: i, text });
  }
  return pages; // [{ pageNumber, text }, ...]
}
```

chunker.js

```
export function chunkText(pages, filename, chunkSize = 500, overlap = 50) {
  const chunks = [];
  for (const { pageNumber, text } of pages) {
    const words = text.split(/\s+/);
    for (let i = 0; i < words.length; i += (chunkSize - overlap)) {
      const slice = words.slice(i, i + chunkSize).join(' ');
      if (slice.trim().length === 0) continue;
      chunks.push({
        id: `${filename}-p${pageNumber}-${i}`,
        text: slice,
        source: filename,
        page: pageNumber,
      });
    }
  }
  return chunks;
}
```

7. Component 2 — Embedding & Vector Store

Owner: Person A.

embedder.js

```
import { pipeline } from '@xenova/transformers';

let embedderPromise;
function getEmbedder() {
  if (!embedderPromise) {
    embedderPromise = pipeline('feature-extraction', 'Xenova/all-MiniLM-L6-v2');
  }
  return embedderPromise;
}
```

```

}

export async function embedText(text) {
  const embedder = await getEmbedder();
  const output = await embedder(text, { pooling: 'mean', normalize: true });
  return Array.from(output.data); // 384-dim vector
}

```

indexedDbStore.js

```

import { openDB } from 'idb';

const dbPromise = openDB('rag-store', 1, {
  upgrade(db) {
    db.createObjectStore('chunks', { keyPath: 'id' });
  },
});

export async function saveChunk(chunk, vector) {
  const db = await dbPromise;
  await db.put('chunks', { ...chunk, vector });
}

function cosineSim(a, b) {
  let dot = 0, na = 0, nb = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i]*b[i]; na += a[i]*a[i]; nb += b[i]*b[i];
  }
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
}

export async function searchTopK(queryVector, k = 5) {
  const db = await dbPromise;
  const all = await db.getAll('chunks');
  return all
    .map(c => ({ ...c, score: cosineSim(queryVector, c.vector) }))
    .sort((a, b) => b.score - a.score)
    .slice(0, k);
}

```

8. Component 3 — On-Device LLM (WebLLM)

Owner: Person B. Runs entirely in a Web Worker so the UI never freezes during generation.

webllmWorker.js

```
import * as webllm from '@mlc-ai/web-llm';

let engine;

self.onmessage = async (e) => {
  const { type, payload } = e.data;

  if (type === 'init') {
    engine = await webllm.CreateMLCEngine(
      payload.modelId, // e.g. "Phi-3.5-mini-instruct-q4f16_1-MLC"
      { initProgressCallback: (p) => self.postMessage({ type: 'progress', payload: p }) }
    );
    self.postMessage({ type: 'ready' });
  }

  if (type === 'generate') {
    const stream = await engine.chat.completions.create({
      messages: payload.messages,
      stream: true,
    });
    for await (const chunk of stream) {
      const delta = chunk.choices[0]?.delta?.content || '';
      self.postMessage({ type: 'token', payload: delta });
    }
    self.postMessage({ type: 'done' });
  }
};
```

webllmClient.js (main thread)

```
const worker = new Worker(new URL('./webllmWorker.js', import.meta.url), { type: 'module' });

export function initModel(modelId, onProgress) {
  return new Promise((resolve) => {
    worker.onmessage = (e) => {
      if (e.data.type === 'progress') onProgress(e.data.payload);
      if (e.data.type === 'ready') resolve();
    };
    worker.postMessage({ type: 'init', payload: { modelId } });
  });
}

export function generate(messages, onToken, onDone) {
  worker.onmessage = (e) => {
    if (e.data.type === 'token') onToken(e.data.payload);
    if (e.data.type === 'done') onDone();
  };
  worker.postMessage({ type: 'generate', payload: { messages } });
}
```

Model IDs to use with WebLLM's prebuilt model list:

- Primary: **Phi-3.5-mini-instruct-q4f16_1-MLC** (~2.2GB, best quality/size tradeoff)
- Fallback: **Llama-3.2-1B-Instruct-q4f16_1-MLC** (~800MB, use if WebGPU/VRAM is limited)

9. Component 4 — RAG Orchestration

Owner: Person B, integrated with Person A's retrieval and Person C's UI.

ragPipeline.js

```
import { embedText } from '../embeddings/embedder';
import { searchTopK } from '../vectorstore/indexedDbStore';
import { generate } from '../llm/webllmClient';

const SYSTEM_PROMPT = `You are a study assistant. Answer ONLY using the provided context. If the answer is not in the context, say you don't know. Cite the source page number(s) you used.`;

export async function askQuestion(question, onToken, onDone) {
  const qVector = await embedText(question);
  const topChunks = await searchTopK(qVector, 5);

  const context = topChunks
    .map(c => `[Source: ${c.source}, page ${c.page}]\n${c.text}`)
    .join('\n\n');

  const messages = [
    { role: 'system', content: SYSTEM_PROMPT },
    { role: 'user', content: `Context:\n${context}\n\nQuestion: ${question}` },
  ];

  generate(messages, onToken, onDone);
  return topChunks; // for showing citations in UI
}
```

10. Component 5 — UI / Chat Interface

Owner: Person C.

- **DocUploader.jsx** — drag-and-drop file input, calls parser + chunker + embedder, shows a progress bar while indexing.
- **ChatWindow.jsx** — message list + input box, calls ragPipeline.askQuestion, renders streamed tokens as they arrive.
- **MessageBubble.jsx** — renders answer text plus small citation chips (source file + page).
- **OfflineIndicator.jsx** — listens to `navigator.onLine` and window online/offline events, shows a badge: "Offline — running 100% on-device".
- **ModelSelector.jsx** — lets user pick Phi-3.5-mini vs Llama-3.2-1B depending on device speed; shows first-load download progress (model is cached by the browser afterward).

11. Optional Cloud Layer (Supabase)

Used ONLY for non-AI support features, to keep the on-device privacy pitch intact:

- Supabase Auth — simple email/OAuth login, optional.

- Supabase Postgres table `documents(id, user_id, title, uploaded_at)` — stores metadata only, never chunk text, never embeddings, never chat history.
- This lets a user see a list of "documents I've uploaded" across devices without any private content leaving their browser.

Important: Do not sync IndexedDB content or chat history to Supabase — doing so would undermine the entire on-device/privacy pitch of the project.

12. Deployment Guide

- Build: ``npm run build`` (Vite outputs a static ``dist/`` folder).
- Host on Vercel or Netlify — both give free HTTPS, which is required for WebGPU and the Cache API to work.
- Do not self-host model weights — WebLLM pulls Phi-3.5/Llama weights from Hugging Face/MLC-AI's CDN on first load, then the browser Cache API stores them locally for all future visits (fully offline after that).
- If using Supabase, add environment variables (`SUPABASE_URL`, `SUPABASE_ANON_KEY`) in the hosting platform's dashboard — never commit these to the repo.
- Test on at least 2 different browsers (Chrome/Edge have the best WebGPU support as of mid-2026; confirm Firefox/Safari WebGPU status before the demo since support varies).

13. Team Role Split (3 People)

Person A — Retrieval Pipeline

- Document upload + parsing (`pdf.js`, `mammoth.js`)
- Chunking logic
- `Transformers.js` embedding integration
- IndexedDB vector store + cosine similarity search
- Deliverable: working ``uploadDoc(file)`` → chunks → embeddings → stored, and ``searchTopK(query)`` → top-k chunks

Person B — LLM & Generation

- WebLLM integration inside a Web Worker
- Prompt template design (context + question → answer with citations)
- Streaming response handling
- Model loading/caching UX (first-time download progress bar)
- RAG orchestration (`ragPipeline.js`)

Person C — UI/UX, Integration & Deployment

- React app shell, chat UI, upload UI, offline-mode indicator
- Wiring Person A + Person B's modules together
- Supabase auth/metadata sync (optional layer)
- Deployment to Vercel/Netlify, cross-browser/device testing
- Demo script and the live "turn off wifi" moment

14. Build Timeline (24–36 hr hackathon)

Hours	Milestone
0–4	Repo setup; WebLLM hello-world (B); PDF upload + chunking (A); React shell (C)
4–10	Embedding + vector search working (A); model loads and generates text (B); chat UI skeleton (C)
10–16	Full RAG loop wired end-to-end: upload -> ask -> retrieve -> generate (all three integrating)
16–22	Streaming responses, offline indicator, citation UI, error handling for slow devices
22–28	Deploy; test on 2–3 different laptops/browsers; fix WebGPU fallback issues
28–32	Demo rehearsal — the "wifi off" moment; prep answers for judge Q&A
Buffer	Keep the last couple hours free — WebGPU quirks across browsers are the most likely late breakage

15. Testing Checklist

- Upload a multi-page PDF and confirm chunks appear in IndexedDB (check via DevTools > Application > IndexedDB).
- Ask a question with wifi ON — confirm correct retrieval + generation.
- Turn wifi OFF (after model + page are cached) — confirm identical behavior.
- Test on a device without a discrete GPU — confirm fallback model works or gives a clear message.
- Test with an empty/no-document state — app should not crash, should prompt user to upload first.
- Confirm citations shown in the answer match the actual retrieved chunk's source/page.
- Confirm no network requests are fired during the generate() call (check DevTools > Network tab during inference).

16. Demo Script for Judges

- 1. State the problem: AI study tools require internet and send your private notes to a server.
- 2. Upload a real PDF of notes live.
- 3. Ask a question, get a streamed answer with citations.
- 4. Turn off wifi on stage, ask a second question — show it works identically.
- 5. Show DevTools Network tab (empty during inference) as proof, if time allows.
- 6. Mention the adaptive model-size selector as a scalability/accessibility point.

17. Known Risks & Mitigations

Risk	Mitigation
WebGPU not supported/enabled on judges' laptop	Ship the 1B fallback model; also pre-record a backup demo video
First model load is slow (multi-GB download)	Preload/cache the model on your own demo laptop before presenting; don't rely on venue wifi
Browser tab memory limits with large PDFs	Cap chunk count / document size for the hackathon build; show a friendly limit message
Safari/Firefox WebGPU gaps	Demo on Chrome/Edge; mention browser support as a known limitation, not a blocker
Team wants a desktop app instead of browser	Swap WebLLM for Ollama + a local API call from an Electron/Tauri shell — architecture in sec

18. Appendix — Prompt Templates & Config

18.1 RAG system prompt

You are a study assistant. Answer ONLY using the provided context.
If the answer is not in the context, say you don't know — do not guess.
Always cite the source page number(s) you used at the end of your answer.
Keep answers concise and exam-relevant.

18.2 package.json — key dependencies

```
{
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "@mlc-ai/web-llm": "latest",
    "@xenova/transformers": "^2.17.0",
    "pdfjs-dist": "^4.0.0",
    "mammoth": "^1.7.0",
    "idb": "^8.0.0",
    "@supabase/supabase-js": "^2.45.0"
  }
}
```

18.3 Chunk record schema (IndexedDB)

```
{
  id: string,           // e.g. "notes.pdf-p3-500"
  text: string,         // chunk text
  source: string,       // original filename
  page: number,        // page number
  vector: number[384]  // embedding vector
}
```

18.4 Notes for the local model assisting this build

When generating code against this document: keep all AI inference (embedding + LLM generation) client-side using the libraries specified in section 3. Do not introduce a backend API for embedding or generation — that would break the on-device requirement that is central to this project's theme. Cloud calls are permitted only for the Supabase auth/metadata layer described in section 11.

19. OSDHack 2026 Compliance & Model Documentation

This section follows the OSDHack 2026 Resource Guide's "Simple Project Notes to Include" template exactly, so it can be copied directly into your submission writeup. Fill in the blank benchmark cells after testing on your own build machine.

19.1 Core rule check

- **Theme:** On-Device AI.
- **Core rule:** main AI functionality (embedding + retrieval + generation) runs entirely client-side, in-browser, via WebGPU/WASM — satisfied by design (see Section 2).
- **Cloud support used:** Supabase, limited strictly to auth + document-title metadata sync — matches the rulebook's allowed list (hosting, auth, dashboards, data sync). No AI compute happens in the cloud.

19.2 Models vs. Libraries — explicit distinction

Per the rulebook, a bare architecture import (e.g. an empty transformer stack) does not count as a local model. Both models used in this project are specific, named, downloadable weights artifacts, satisfying the rule:

- **Phi-3.5-mini-instruct-q4f16_1-MLC** — a specific quantized checkpoint hosted on Hugging Face / MLC-AI, pulled and run by WebLLM. Not a generic architecture.
- **Xenova/all-MiniLM-L6-v2** — a specific ONNX weights repository on Hugging Face, pulled and run by Transformers.js. Not a generic architecture.

19.3 Per-model documentation table

Field	LLM (Generation)	Embedding Model
Model name	Phi-3.5-mini-instruct (q4f16_1-MLC build)	all-MiniLM-L6-v2 (Xenova ONNX build)
Source link	huggingface.co/mlc-ai/ Phi-3.5-mini-instruct-q4f16_1-MLC	huggingface.co/Xenova/ all-MiniLM-L6-v2
License	MIT (Phi-3.5 base license)	Apache 2.0
File format	MLC-compiled q4f16 (WebGPU)	ONNX (quantized)
Approx. size	~2.2 GB	~25 MB
Local runtime	WebLLM (WebGPU, Web Worker)	Transformers.js (WASM/WebGPU)
Fine-tuned / converted / quantized	Used pre-quantized MLC build as-is (no further fine-tuning)	Used pre-quantized ONNX build as-is
Target hardware	Laptop/desktop browser with WebGPU (Chrome/Edge); CPU fallback via smaller model	Any browser, CPU or WebGPU
Performance (fill in after testing)	_____ tokens/sec on test laptop	_____ ms per chunk embedded

19.4 Fallback model (if primary is too heavy on demo hardware)

Field	Value
Model name	Llama-3.2-1B-Instruct (q4f16_1-MLC build)
Source link	huggingface.co/mlc-ai/Llama-3.2-1B-Instruct-q4f16_1-MLC
License	Llama 3.2 Community License (custom, redistribution terms apply)
File format	MLC-compiled q4f16 (WebGPU)
Approx. size	~800 MB
Local runtime	WebLLM (WebGPU, Web Worker)
Notes	Swap-in replacement selectable via ModelSelector.jsx when WebGPU/VRAM is limited

19.5 License note

Double-check current license text on each model's Hugging Face page before submission, since license terms can change between model card revisions. Always link the exact model card, not just the base model family, in your final writeup.

19.6 Official event resources

- Organizer: Open Source Developers Community (OSDC)
- Support channels: OSDHack Discord and OSDHack Unstop listing
- Reference libraries confirmed against the rulebook's Quick Links Directory: WebLLM, Transformers.js, ONNX Runtime Web, Hugging Face Models